

Análise Técnica — InkFlow

Escopo: todo o código Dart de `lib/` e `test/`, `pubspec.yaml`, as migrations em `supabase/migrations/` e a documentação em `Docs/`.

Ferramentas usadas: `flutter pub get`, `flutter analyze lib test`, `flutter test`.

Estado ao final da intervenção: analyzer sem nenhuma issue; 63 testes passando (eram 12).

Sumário executivo

O aplicativo tinha uma interface bem acabada e um fluxo de autenticação sólido, mas três problemas estruturais dominavam todo o resto:

1. **Não existia controle de acesso no banco versionado.** O código de chat e de busca baixava dados de todos os usuários e filtrava no aparelho. Qualquer portador da chave anônima conseguia ler conversas alheias e o CPF de qualquer tatuador.
2. **Quatro dos dez requisitos funcionais eram telas de mentira.** Anamnese, dashboard, lembretes e propostas no chat não persistiam nada — e a tela de anamnese ainda afirmava proteger dados de saúde sob a LGPD.
3. **O projeto não estava sob controle de versão.** Sem histórico, sem branches, sem backup.

Os itens 1 e 2 foram tratados. O item 3 permanece em aberto por decisão sua.

1. Segurança e LGPD

1.1 Ausência total de RLS (*corrigido*)

A única migration existente (`001_rf02_artist_profile.sql`) criava colunas e o bucket de portfólio. Não havia uma linha sequer de política para `profiles`, `messages` ou `appointments`.

O chat era a prova mais clara disso: o stream trazia a tabela `messages` inteira e o filtro por interlocutor acontecia em Dart. A inbox fazia exatamente o mesmo.

Correção: criada a migration `002_rls_policies.sql`, idempotente, com:

Objeto	Regra
<code>profiles</code>	Cada usuário lê e escreve apenas a própria linha. <code>UPDATE</code> revogado em <code>cpf</code> e <code>rating</code> .
<code>messages</code>	<code>SELECT</code> apenas para remetente ou destinatário; <code>INSERT</code> só em nome próprio (<code>WITH CHECK</code> em <code>sender_id</code>).
<code>appointments</code>	<code>SELECT</code> para o tatuador e o cliente da sessão; escrita apenas pelo tatuador.
<code>artist_directory</code> (view)	Vitrine pública de tatuadores com lista explícita de colunas não sensíveis.
<code>chat_directory</code> (view)	Nome e avatar apenas de quem já tem histórico de mensagens com o usuário autenticado.

Ação pendente sua: a migration precisa ser executada no SQL Editor do Supabase, depois de `001`. Enquanto isso não acontecer, o banco continua aberto.

1.2 Vazamento de PII nas consultas (*corrigido*)

`select()` sem lista de colunas retorna tudo — inclusive `cpf`, `date_of_birth`, `guardian_cpf` e `phone`. O padrão aparecia em `search_screen`, `scheduleProvider` e `todayAppointmentsProvider`.

Correção: toda leitura de terceiros passou a usar as views `artist_directory` e `chat_directory`; a leitura do próprio perfil usa uma lista explícita de colunas em `ProfileRepository`.

1.3 Filtro de conversa movido para o servidor (*corrigido*)

O Realtime do Supabase aceita apenas um filtro de igualdade por stream, o que tornava impossível expressar "remetente = eu OU destinatário = eu" no servidor — daí o filtro em Dart.

Correção: a coluna gerada `conversation_key` (`least(sender,receiver) || ':' || greatest(sender,receiver)`) dá uma chave estável por conversa, e o app assina exatamente uma.

1.4 Oráculo de CPF (*corrigido*)

`isCpfRegistered` consultava `profiles` por CPF **antes** do login. Sem RLS, isso permitia a qualquer pessoa com a chave anônima testar se um CPF estava cadastrado.

Correção: a pré-verificação foi removida. A duplicidade agora vem de um índice único (`profiles_cpf_key`) e de um trigger que levanta `cpf_already_registered`, traduzido no cliente por `AuthRepository.parseDuplicateField`.

1.5 CPF em metadata editável (*corrigido*)

`signUp` enviava `cpf`, `date_of_birth` e `guardian_cpf` dentro de `data:`. O metadata do Supabase é editável pelo próprio usuário via `updateUser` — ou seja, o usuário podia reescrever o próprio CPF.

Correção: o trigger `handle_new_user()` copia esses campos para `profiles` (onde o `UPDATE` já está revogado) e em seguida os remove de `raw_user_meta_data`.

1.6 Troca de senha sem reautenticação (*corrigido*)

`change_password_screen` chamava `updateUser(password:)` direto, sem pedir a senha atual. Um dispositivo desbloqueado virava takeover de conta. A política era apenas "mínimo 6 caracteres".

Correção: a senha atual passou a ser obrigatória e é revalidada antes da troca; a força da nova senha é verificada por `PasswordPolicy`.

1.7 Anamnese afirmava proteção inexistente (*corrigido*)

O cabeçalho exibia "Dados protegidos e criptografados (LGPD)", mas o envio era um `Future.delayed`. Nada era gravado — o analyzer confirmava que `_signature`, `_allergyDetails`, `_medicationDetails` e `_diseaseDetails` nunca eram lidos. Dado de saúde é dado sensível na LGPD, e afirmar proteção que não existe é exposição jurídica.

Correção: a alegação foi removida e substituída por um aviso explícito de pré-visualização, deixando claro que nada é enviado nem armazenado.

1.8 `.env` empacotado como asset (*aceito com ressalva*)

O arquivo está listado em `pubspec.yaml` e, num build web, fica servido em `/assets/.env`. Hoje contém apenas a chave anônima, que é pública por design, e o `flutter_dotenv` exige que o arquivo seja asset para conseguir lê-lo.

Risco residual: qualquer segredo futuro colocado nesse arquivo vaza junto. Se surgir a necessidade de um segredo real, ele precisa ir para uma Edge Function, nunca para o `.env` do app.

2. Funcionalidades que aparentavam funcionar

Conforme sua decisão, estas telas foram **sinalizadas como em desenvolvimento** em vez de implementadas agora. O componente `UnderDevelopmentBanner` foi criado para isso.

Tela	Situação anterior	Situação atual
Anamnese (RF07)	Alegava proteção LGPD, não gravava nada	Banner de pré-visualização; alegação removida
Dashboard (RF10)	<code>Future.delayed</code> + seis meses fixos; "Melhor Mês" hardcoded	Banner de dados de demonstração
Lembretes (RF09)	Lista fixa de três clientes; <code>ScenarioSwitcher</code> de protótipo visível	Passou a ler agendamentos reais; banner sobre o disparo automático
Notificações	<code>_mockAppointments</code> e um <code>Timer.periodic</code> que apagava o estado "lida" a cada minuto	Derivadas da agenda real; banner sobre push
Propostas no chat (RF04/RF05)	Card inalcançável; botão exibia "Requer integração com banco"	Sinalizado como em desenvolvimento
Favoritos	Apenas o estado vazio	Banner de funcionalidade indisponível

Além disso, o menu de perfil tinha quatro itens que prometiam telas diferentes e entregavam a mesma: `/activate-artist`, `/artist-portfolio` e `/artist-pricing` apontavam todos para `ProfileSetupScreen`, e `/artist-quotes` para a `InboxScreen`. As rotas duplicadas foram removidas e o menu consolidado.

O chat também exibia nome e foto do interlocutor hardcoded (uma URL do Unsplash e o texto "Eduardo (Cliente)"), ignorando o identificador recebido. Agora resolve o contato real pela view `chat_directory`.

3. Bugs concretos

Bug	Consequência	Situação
<code>DateFormat('MMM', 'pt_BR')</code> sem <code>initializeDateFormatting</code>	<code>LocaleDataException</code> na home do cliente assim que a agenda carregasse	Corrigido em <code>main.dart</code>
<code>currentUserProvider</code> era um <code>Provider</code> simples lendo <code>auth.currentUser</code> uma vez	Após logout/login continuava devolvendo o usuário antigo	Corrigido: deriva de <code>onAuthStateChanged</code>
<code>routerProvider</code> observava <code>authStateProvider</code>	Cada evento de auth — inclusive o <code>tokenRefreshed</code> horário — reconstruía o <code>GoRouter</code> e zerava a pilha	Corrigido com <code>refreshListenable</code>
<code>queryParameters['artistId'] ?? '1'</code>	'1' não é UUID; o insert era rejeitado pelo banco	Corrigido: redireciona para <code>/inbox</code> quando falta o parâmetro
<code>s['time']</code> as <code>String</code> e afins na agenda	Crash com qualquer registro de campo nulo	Corrigido pelo modelo <code>Appointment</code> com parsing tolerante
<code>currentUser!.id</code> na agenda	Crash com sessão expirada	Corrigido
Conflito de horário validado só no cliente	Duas gravações concorrentes passavam pelas duas	Corrigido: <code>constraint EXCLUDE USING gist</code> no banco
Campos de horário em texto livre	Digitar 9h fazia a conversão devolver 0 silenciosamente	Corrigido: <code>showTimePicker</code> + utilitário <code>TimeSlot</code>
<code>emailController</code> não descartado no diálogo de recuperação de senha	Vazamento de memória ao cancelar	Corrigido com <code>try/finally</code>
<code>context.go('/home')</code> manual junto do <code>redirect</code> do router	Navegação dupla	Corrigido
Botão de busca decorativo	A filtragem já ocorria a cada tecla; o botão só ligava uma flag de erro	Corrigido: filtros aplicados explicitamente
Upload parcial de portfólio silencioso	Perfil ativado com portfólio incompleto e sem aviso	Corrigido: a falha agora interrompe e informa

4. Arquitetura

4.1 Camada de dados (*corrigido*)

Só `AuthRepository` havia sido extraído. Todo o resto chamava `Supabase.instance.client` direto — dentro dos providers e, pior, dentro das telas (`chat_screen._sendMessage`, `schedule_screen._handleSave`, `edit_profile._saveProfile`, `change_password._updatePassword`, `profile_screen._signOut`, `profile_setup._activateProfile`). Era exatamente isso que tornava 90% do app impossível de testar.

Correção: repositórios por feature — `ProfileRepository`, `ChatRepository`, `AppointmentRepository`, `ArtistDirectoryRepository` — e um `supabaseClientProvider` central, que permite injetar um cliente falso nos testes.

4.2 Modelos tipados (*corrigido*)

Tudo trafegava como `Map<String, dynamic>` com acesso por string e casts manuais; um erro de digitação em nome de coluna só aparecia em produção.

Correção: criados `UserProfile`, `ArtistSummary`, `ChatMessage`, `Conversation`, `ChatContact`, `Appointment`, `NewAppointment` e `AppNotification`, todos com fábricas tolerantes a linhas incompletas.

4.3 Providers duplicados (*corrigido*)

`userRoleProvider`, `userProfileProvider`, `clientProfileProvider` e a consulta embutida no `scheduleProvider` buscavam todos a mesma linha de `profiles`, com regras de fallback diferentes entre si. Foram consolidados em `userProfileProvider`.

4.4 Pontos ainda abertos

- **Telas monolíticas.** `search_screen` tem cerca de 700 linhas, `chat_screen` 620 e `register_screen` 570, cada uma num único `State`, sem quebra em widgets. Na busca, o `setState` no topo re-renderiza o grid inteiro a cada tecla.
- **Design tokens duplicados.** Existem `InkFlowColors.error/success/warning`, mas os mesmos valores aparecem como literais (`0xFFEF4444`, `0xFF10B981`, `0xFF6B7280`) em quase todos os arquivos.
- **Tema de input incoerente.** O `inputDecorationTheme` global foi desenhado para fundo escuro enquanto o `scaffoldBackgroundColor` é claro — por isso cada tela reimplementa o próprio `InputDecoration` do zero. Não há tema escuro nem `ThemeMode`.
- **Navegação inconsistente.** `AppBottomNav` usa `context.go` (descarta a pilha) enquanto várias telas usam `context.push`, o que torna o comportamento do botão "voltar" imprevisível.

5. Testes

Antes: 12 testes cobrindo `ArtistHomeScreen`, `PrivacyScreen` e dois auxiliares de erro. Nenhum teste para login, cadastro, conflito de agenda, anamnese ou guards de rota. O `widget_test.dart` era literalmente uma cópia de um teste já existente em `auth_repository_test.dart`.

Depois: 63 testes. O arquivo duplicado foi removido e a lógica de router e de parsing foi extraída para funções puras justamente para poder ser testada.

Arquivo	Cobre
<code>register_rules_test.dart</code>	Validação de CPF e cálculo de idade (núcleo do RF01)
<code>router_guard_test.dart</code>	Redirecionamentos de autenticação e parsing do parâmetro de chat
<code>time_slot_test.dart</code>	Parsing, normalização e detecção de sobreposição de horários
<code>appointment_test.dart</code>	Parsing tolerante de linhas e lógica de data/slot
<code>artist_summary_test.dart</code>	Fallbacks de perfil público e imagem de capa
<code>chat_test.dart</code>	Chave de conversa, mensagens, prévia da inbox e contato
<code>password_policy_test.dart</code>	Regras de força de senha
<code>notifications_rules_test.dart</code>	Derivação de avisos a partir da agenda
<code>auth_repository_test.dart</code>	Tradução das mensagens de duplicidade vindas do trigger

Ainda aberto: não há `mocktail` / `mockito` nem um duplo do Supabase, então os repositórios são testados apenas pelas partes puras (parsing, chaves, regras). Testes de integração exigiriam uma instância de teste ou um fake do `SupabaseClient`.

6. Infraestrutura e qualidade

6.1 Controle de versão (*em aberto — decisão sua*)

Não existe pasta `.git` no projeto. Sem histórico, sem branches, sem backup: uma pasta apagada por engano leva tudo. **Esta continua sendo a recomendação de maior impacto e menor custo do relatório.**

Consequência direta: a limpeza do lixo versionado (abaixo) foi adiada, porque remover arquivos sem controle de versão é irreversível.

6.2 Lixo no repositório (*em aberto*)

`telas.rar`, a pasta `telas/` com 15 PNGs, `package.json` / `package-lock.json` com `puppeteer-core` (usado apenas para gerar screenshots), `.idea/`, `inkflow.iml` e a pasta `.github/` contendo só resíduo de uma ferramenta de upgrade de Java — nenhum workflow de CI.

6.3 Configuração (*corrigido*)

- `.gitignore` ignorava `pubspec.lock` via `*.lock`. Para um aplicativo — diferente de um package — o lockfile deve ser versionado, senão os builds não são reproduzíveis. Corrigido, com o motivo registrado em comentário.

- `analysis_options.yaml` referenciava `prefer_const_widgets`, que não é um lint válido e gerava `undefined_lint`. Corrigido, com regras mais estritas no lugar.

6.4 Analyzer (corrigido)

Eram 115 issues: cerca de 90 usos de `withOpacity` (depreciado), além de `RadioListTile.groupValue / onChanged`, `DropDownButtonFormField.value` e `Supabase.initialize(anonKey:)` — todos depreciados. Nenhum quebrava naquele momento, mas todos quebrariam numa atualização futura do Flutter. Hoje o analyzer roda limpo.

6.5 Dependências (corrigido)

`cached_network_image` era declarada e documentada no README, mas todas as telas usavam `Image.network / NetworkImage` cru. Passou a ser efetivamente usada por `NetworkImageWithFallback`, que também trata erro de carregamento. `image_cropper` era declarada e nunca usada — foi removida.

`flutter_lints` continua na versão 3 (a 6 já saiu). Atualizar traria novas regras, e por isso ficou fora deste ciclo.

6.6 Acessibilidade e internacionalização (em aberto)

- Nenhum `Semantics`, praticamente nenhum tooltip.
- Tamanhos de fonte fixos em 10–11 px que ignoram a escala de texto do sistema.
- Contrastes que reprovam no WCAG — o texto de termos no login usa `Color(0x33FFFFFF)`, cerca de 20% de opacidade.
- `supportedLocales` declara `en_US`, mas 100% das strings estão fixas em português, com mistura de português europeu ("ecrã", "utilizador", "aceder", "Escreve uma mensagem...") e brasileiro.

7. Recomendação de próximos passos

1. **Inicializar o git** e fazer o primeiro commit. Sem isso, todo o resto é frágil.
2. **Executar** `002_rls_policies.sql` no Supabase. As correções de segurança do código pressupõem que ela esteja aplicada.
3. **Limpar o lixo do repositório** (item 6.2), o que fica seguro assim que o passo 1 estiver feito.
4. **Decidir o destino das telas mock**: implementar de verdade ou remover do menu. O banner é uma medida provisória, não um destino.
5. **Adicionar CI** rodando `flutter analyze` e `flutter test` a cada push.
6. **Quebrar as telas monolíticas** e unificar os design tokens.
7. **Acessibilidade e i18n**, quando o produto estiver mais estável.